Title:

Terraform

StarNote

1. What is Infrastructure as Code.

Practice which enables developers and operations teams to provision and manage infrastructure through code & instruction rather than manually configuring the process.

a) Automation: ✓

Iac allows to perform infrastructure tasks automatically via code.

- No manual interventions
- Less chances of error
- Rapid
- Streamlines repeatitive tasks.

Eg. Using Terraform to create EC2 instead of using Aws.

b) Declarative & Imperative ✓

- → tell tool what to do and desired result

tool - configures and determines the necessary steps.

tell tool how to do it by specifying the commands ① or steps ② to achieve the result.

StarNote

c) Version Control: ✓

Tracking changes made to code over time.

Supports rollback to older builds incase inconsistencies in code resulting in problem for newer builds.

d) Idempotence: ✓

Output of fixed ① configuration of code would produce exact same intended result .... ② over and over again.

Eg. Tfm code configured to create _x86 EC2 wont create Arm64 EC2

No manual config mistake.

e) Consistency: IaC creates identical environment with consistent configurations.

Reproducibility: recreation of envirn is very easy using same code.

StarNote

(2) Benefits of IaC:

- ✓ Speed and Efficiency without human intervention.
- ✓ Cost-effective: by automation.
- ✓ (**) Disaster Recovery: Recreation of environment is rapid in events of loss.
- ✓ Scalability: IaC can scale infra-automatically

(3) Practice: Terraform Code to provision EC2.

- Configure Aws IAM Role (AwsEC2full access)
- Attach to host EC2.... Install Terraform
- Mkdir terraform_cd.... write main.tf
- terraform init, plan, apply.

StarNote

4. Challenges:

→ Complexity in large-scale deployment: Managing large and complex infrastructure with IaC

→ (State Management:)

→ Security: Storing sensitive data exposed in codebase can be a security risk. Solution: using Vault.

StarNote

II. Terraform.

1. What is terraform?

Terraform is an open source ① Infrastructure as Code (IaC) tool created by HashiCorp. ②

It allows us to ① define, ② provision, and ③ manage cloud resources using a declarative config language. ✓

How terraform work:

- **Write**: (terraform init); configuration file (.tf) and initialize working directory to download the necessary provider plugins. ✓ (creates environment)
- **Plan**: (terraform plan); previews changes it will make (create, update, destroy) without actually touching infrastructure
- **Apply**: (terraform apply); Terraform executes the planed changes, provision or updating resources in cloud env.

StarNote

Terraform Vs Ansible:

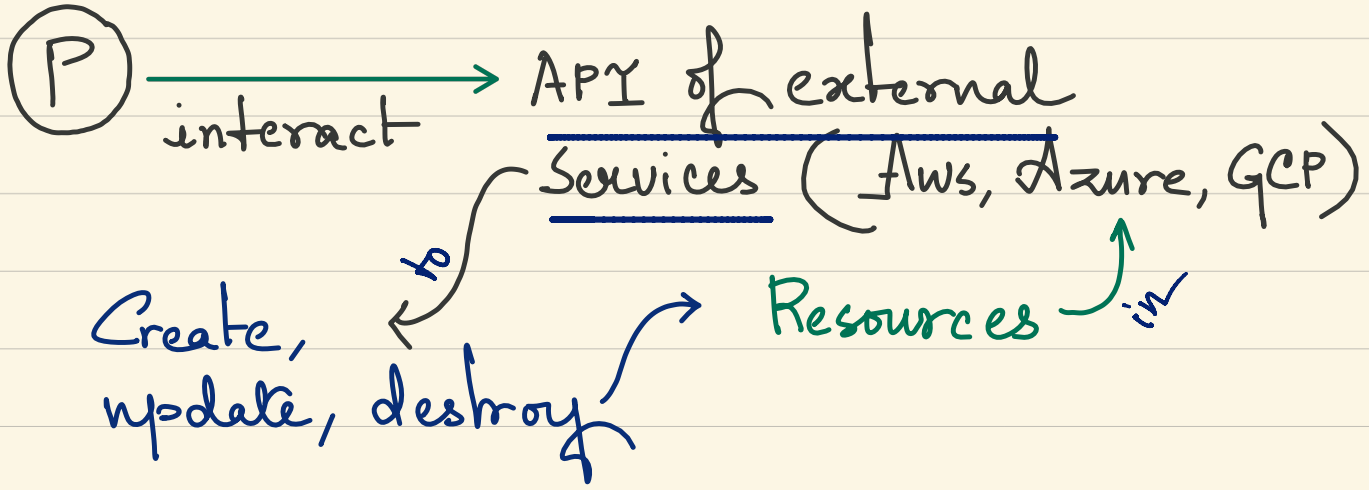
Terraform infrastructure code tool — used to provision infrastructure via IaC

Ansible is infrastructure configuration tool used to configure and customised infra - after being created.

Aws (Cloud formation) = terraform

StarNote

2. Provider:



Syntax:

```
provider "aws" {
    region : "us-east-1"
    secret_key : "....."
    access_key : "....."
                    }
```

3. Resources: A resource in terraform represents a specific components of our infrastructure

Its defined by: resource block.

```
resource "aws_instance" "name" {
block    parameter      nm
    ami = "ami-0511...."   arg
    instance_type = "t3.micro"
                    }
```

StarNote

4. Attributes: Resource typically include multiple attributes such as:

Argument (human config)

a. ami 'holds the ami Id'
b. instance_type 'holds the type power resource would get'
c. tags 'Special key: value pair attached to resource'

(Input ⟹ Provider API ⟹ Set up Infrastructure

Arg → Data before creating
Computed → Data after creating (Cloud provider API Config)

Computed

provide metadata and details about the resource that are obtained after resource actually exists.

Eg. value = aws_instance.web.public_ip

5. State
- what is Terraform State?

Terraform state file (terraform.tfstate) used to keep track of resources that Terraform manages. Stores metadata about your infrastructure in statefile.

CamNote

Crucial ensuring terraform understand the current state of infrastructure.

Important :-

a) State file ensures terraform (update, created or destroy) decisions code maps configuration real infrastructure

b) State Management :

By default state is locally stored in <terraform.tfstate>
by for larger environments use :

S3 for Aws.

c) Remote State :

Storing statefile remotely is beneficial for collaboration.

StarNote

Terraform State In details:

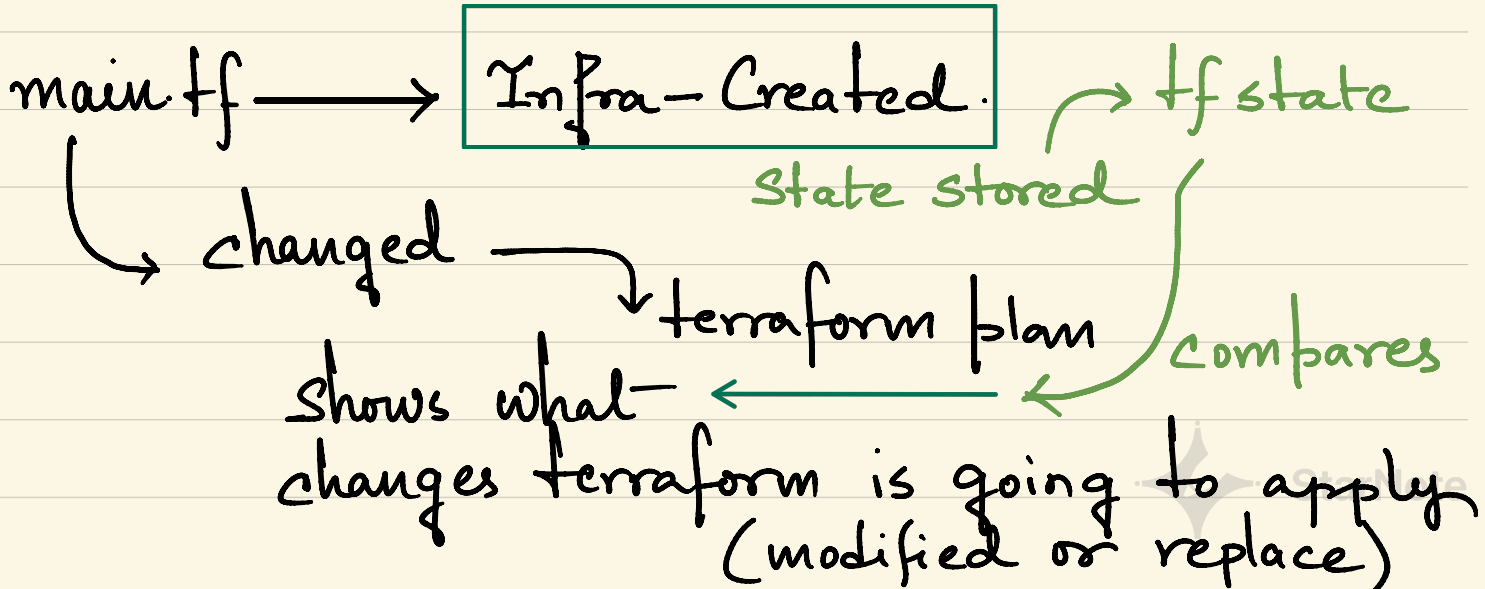
main.tf (Terraform Config)
store what we want to do with our infrastructure

terraform.tfstate (terraform State)
store what terraform knowns about our infrastructure.

Aws infrastructure
(resources running)

Use of State file:
- Resource tracking
- Change detection
- Collaborations.

Example on how stateworks.



State locking

State locking: is a mechanism that prevents multiple-users modify the same state simultaneously (No concurrent changes)

Problem: operations may overlap, conflicting changes, mismatched configuration, leading to state inconsistency

How terraform uses statelocking

- dev A: starts Apply → Terraform acquires lock → shared state
- dev B: tries to Apply → operation awaits completion (A) or runs into lock-related error

lock is released once dev A's task is completed.

backend.tf

```
terraform {
  backend "s3" {
    bucket = "..."
    key    = "..."
    region = "..." }
}
```

Commands:

use_lockfile = true

StarNote

Scenarios :

StarNote

6. Terraform Configuration files: (HCL)

- ✓ Provider Block: (Specifies cloud provider)

```
provider "aws" {
  region = " " }
```

- Resource Block: (Defines what resource to manage)

```
resource "aws_instance" "name" {
  ami = "111
  instance type = " "
}
```

- ✓ Variable block: allows → declare configs; make configurations dynamic and reusable

StarNote

```
variable "instance type" {
    default = "t3.micro"
}
```

- Output block:

Output values to console after running terraform apply

```
output "instance_ip" {
    value = aws_instance.example.
            public_ip.
```

- Data Sources:
  Allows us to retrieve information about existing infrastructure can also be used in configuration.

7. Terraform Workflow:

- init: initializes working directory (→ Sets up environment) downloads necessary provider plugins configures backend if using remote state.

StarNote

- validate: • Syntax & Internal Consistency (invalid argument/ incorrect d-type) checking
  - Attribute & variable checks.
  - Isolated verification checks performed exclusively at local/offline
- plan: Show preview of actions terraform is about to perform and output
  (Create(Addition), modify(changes), remove(deletion).
- apply: Applies actions specified by the plan to the infrastructure
- destroy: Destroys infrastructure maintained by terraform. [Cleanup]

8. Variables:

variables in terraform provides versatility and reusability while configuring.

Syntax: variable "name" {
description = ""
default = "" }

How to use: <var.name>

Ways to override variables:
> commandline: -var="region=val"
> terraform.tfvars file.

9. Modules:

Collection of Terraform configuration files grouped together to create and manage AWS resources

Examples:— 5 EC2 instances with same configuration
→ Create 1 EC2, reuse it 5 times

- Reusability
- Organization
- Consistency
- Easy Maintenance:

StarNote

Structure of Module:

folder: (local)
- ↳ main.tf
- ↳ variables.tf
- ↳ Outputs.tf

(Remote terraform registry)

```
module "vpc" {
source = "./module/
          vpc"
cidr_block = " "
}
```

1. **Root Module:** Main directory where init, apply are called. It calls other modules when required

2. **Child Module:** Module which is called by the root, contains resuable configs EC2, S3, or VPC.

3. **Published or external module:** resuable module obtained from terraform registry.

(Note:)

Avoid storing sensitive information directly in terraform files.

Good practice to use services like Aws Secret Manager, HasiCorp Vault or terraform cloud to securely manage secrets.

StarNote

10. Provisioners & Lifecycle:

Provisioners: allows you to execute scripts or commands on resource after they have been created

- Types of provisioners:
  - remote-exec: Executes commands over SSH
  - local-exec: Executes commands locally

lifecycle:

- prevent_destroy: Prevent accidental destruction of resource.
- ignore_changes: Ignore changes to certain attributes of resource.

StarNote

11. Terraform - Interpolation:

Interpolation in Terraform → inserting the value of a [variable, resource attribute, or expression into a string or configuration

Helps to use dynamic values instead of using fixed / hardcoded values.

old Syntax: ${var.instance_name}

New Syntax: var.instance_name.

Example

```
resource "aws-default-vpc" default {
  id:
}

resource "aws-security-group" my-sg {
  vpc_id = aws_default-vpc.default.id
```

..... dynamic value binding.

StarNote

StarNote